

# PGI LARAVEL

Progiciel de Gestion Intégré pour une PME

– Cahier de charges –

# Plan

1. Présentation du projet
  - 1.1 Objectifs
  - 1.2 Cible
  - 1.3 Responsabilités
2. Description de la solution
  - 2.1 Besoins fonctionnels
  - 2.2 Besoins non fonctionnels
3. Contraintes générales
  - 3.1 Qualité de la solution
  - 3.2 Temps de développement
  - 3.3 Coûts de développement
4. Livrables attendus

# 1.Présentation du projet

Le projet PGI Laravel vise à concevoir et développer une solution web centralisée permettant à une PME de gérer ses activités quotidiennes via un outil unique.

Le système permettra d'unifier la gestion des différents services (ventes, achats, stocks et RH) dans une base de données commune, afin d'améliorer la productivité et la traçabilité des opérations.

## 1.1 Objectifs

- Développer un PGI complet et modulaire pour une PME.
- Centraliser les données liées aux ventes, achats, stocks et RH.
- Réduire la duplication des données et les erreurs de saisie.
- Automatiser les processus métier pour gagner en productivité.
- Offrir une interface intuitive et réactive aux utilisateurs.
- Préparer un hébergement web pour une utilisation multi-utilisateurs.

## 1.2 Cible

- Entreprises de petite ou moyenne taille (PME) souhaitant numériser leur gestion.
- Les utilisateurs internes de l'entreprise : gérants, employés administratifs, responsables RH et commerciaux.
- Le PGI sera principalement utilisé sur poste de travail ou tablette, via un navigateur web.

## 1.3 Responsabilités

Responsabilités principales :

- **Équipe de développement (Kenza chahnaz & Ngueba Anne-Maéra)**
  - Conception de la base de données et des modules métiers (ventes, achats, stocks, RH).

- Développement Front-end (interfaces Laravel Blade) et Back-end (contrôleurs, modèles, API Laravel).
  - Tests fonctionnels et techniques en local via XAMPP.
  - Préparation du déploiement en ligne.
  - Suivi de la qualité du code, documentation et gestion de projet.
- 
- **Encadrant pédagogique - CHERGUI Anis :**
    - Évaluation du projet lors de la soutenance finale.
- 
- **Utilisateur final (PME fictive) :**
    - Définir les besoins du PGI (ventes, achats, stocks, RH).
    - Vérifier que le système fonctionne bien et répond aux attentes.
    - Vérification du bon fonctionnement de la solution en environnement local (XAMPP) puis en ligne, après hébergement.



## 2. Description de la solution

### 2.1 Besoins fonctionnels

| Module / Fonctionnalité  | Description                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------------------|
| Ventes                   | Gestion des clients, création et suivi des commandes, génération des factures.                  |
| Achats                   | Gestion des fournisseurs, enregistrement des bons de commande, suivi des paiements.             |
| Stocks                   | Suivi des entrées et sorties de produits, gestion des niveaux de stock et alertes d'inventaire. |
| Ressources Humaines (RH) | Gestion des employés, des contrats, des présences et congés.                                    |
| Tableau de bord          | Vue d'ensemble des indicateurs clés (ventes, achats, stocks, personnel).                        |

### 2.2 Besoins non fonctionnels

| Catégorie             | Description                                                               |
|-----------------------|---------------------------------------------------------------------------|
| Accessibilité         | Application accessible via un navigateur web.                             |
| Sécurité              | Authentification sécurisée avec rôles et permissions utilisateurs.        |
| Interface utilisateur | Interface ergonomique, responsive et intuitive (Laravel + Blade).         |
| Base de données       | Utilisation d'une base MySQL structurée et relationnelle.                 |
| Hébergement           | Déploiement local via XAMPP, puis hébergement distant sur un serveur web. |

# 3. Contraintes générales

## 3.1 Qualité de la solution

- Le PGI doit être fonctionnel et fiable, sans erreurs critiques.
- L'interface doit être ergonomique, claire et responsive pour une utilisation facile.
- Le code doit être structuré, commenté et documenté pour faciliter la maintenance.
- La base de données MySQL doit être relationnelle et cohérente, avec intégrité des données assurée.
- Les tests doivent être réalisés pour valider toutes les fonctionnalités avant déploiement.

## 3.2 Temps de développement

Le temps alloué pour réaliser le projet est de 3 semaines.

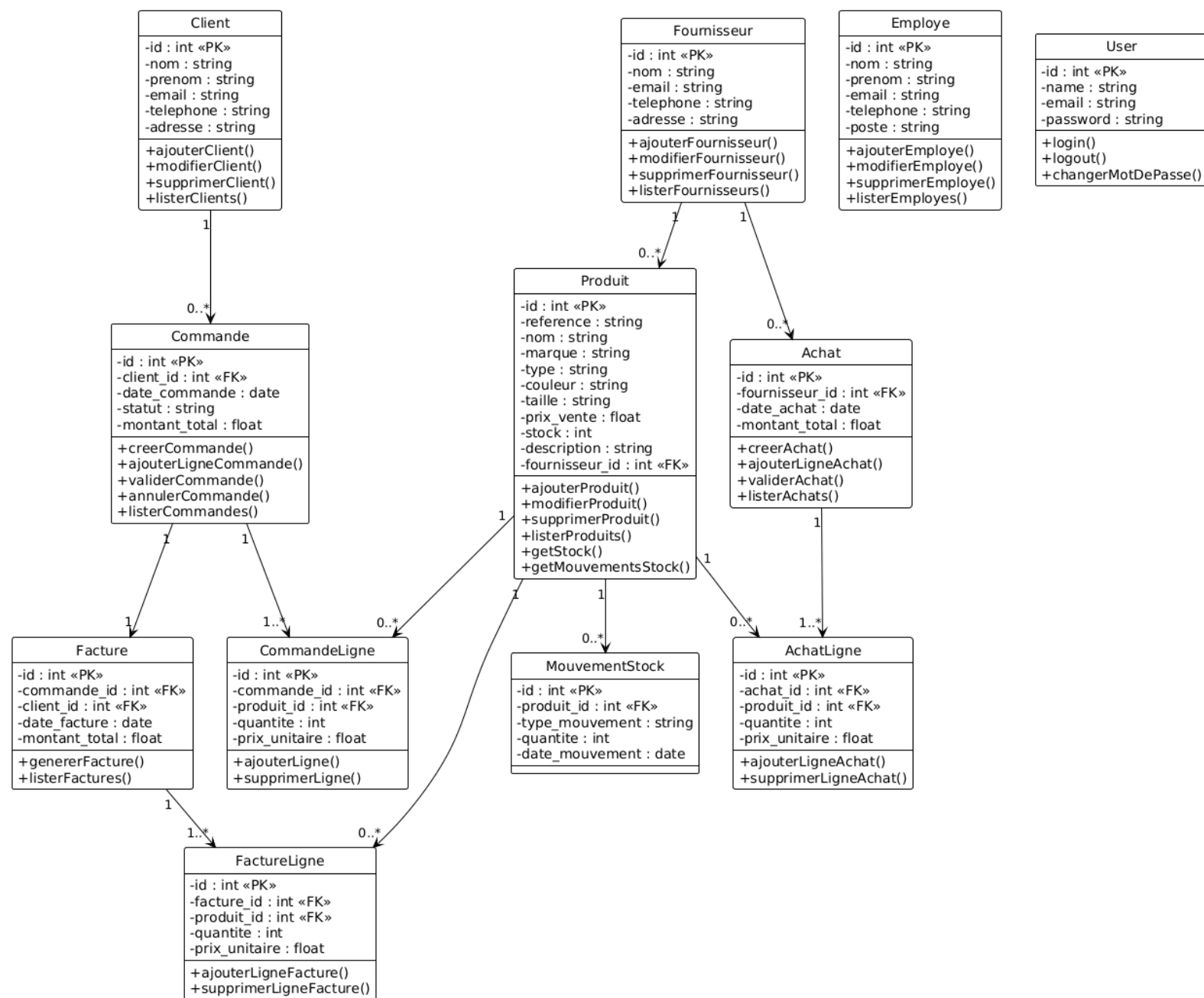
## 3.3 Coûts de développement

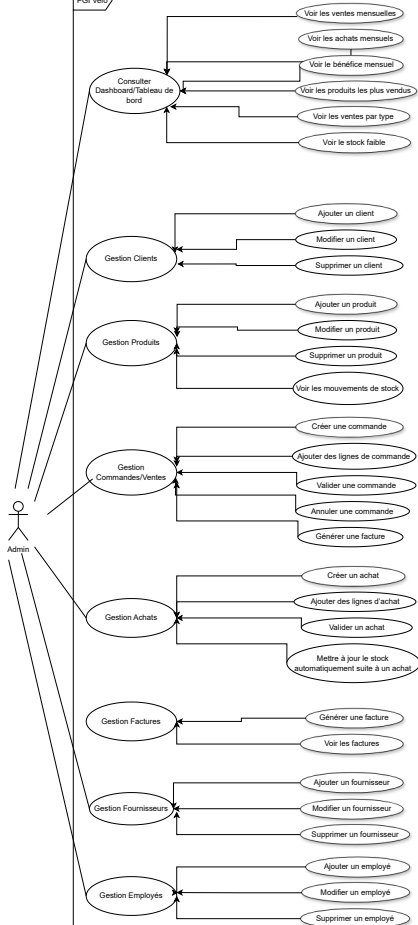
| Ressources / Outils  | Description                                           | Coût estimé |
|----------------------|-------------------------------------------------------|-------------|
| Ordinateur personnel | Machine utilisée pour le développement et les tests   | 0 €         |
| XAMPP                | Serveur local pour tester le PGI (Apache, MySQL, PHP) | 0 €         |
| Laravel              | Framework PHP pour le développement du PGI            | 0 €         |
| MySQL                | Base de données relationnelle                         | 0 €         |
| Hébergement web      | Déploiement en ligne du PGI                           | 0 €         |

Un budget de 0 euros à été alloué pour le développement du projet

## 4. Livrables attendus

| Phase                              | Élément                                        | Livrable                       | Explication courte                          |
|------------------------------------|------------------------------------------------|--------------------------------|---------------------------------------------|
| <b>1. Phase initiale</b>           | Cahier des charges                             | Cahier des charges fonctionnel | Recueille besoins, objectifs et contraintes |
|                                    | Étude de faisabilité                           | Rapport de faisabilité         | Vérifie si le projet est réalisable         |
|                                    | Plan de projet                                 | Plan de gestion de projet      | Organise tâches, calendrier et ressources   |
| <b>2. Analyse</b>                  | Spécifications fonctionnelles générales (SFG)  | Document de SFG                | Vue d'ensemble des fonctionnalités          |
|                                    | Spécifications fonctionnelles détaillées (SFD) | Document de SFD                | Détails des fonctionnalités et scénarios    |
|                                    | Spécifications techniques                      | Dossier technique              | Choix techniques et architecture            |
|                                    | Modèles UML                                    | Diagrammes UML                 | Schémas pour comprendre le système          |
| <b>3. Conception</b>               | Conception architecturale                      | Schéma d'architecture          | Organisation globale du système             |
|                                    | Maquettes / Prototypes                         | Maquettes d'écrans             | Valide l'interface et l'ergonomie           |
| <b>4. Développement &amp; Test</b> | Documentation technique                        | Documentation développeur      | Guide pour utiliser et maintenir le code    |
|                                    | Plan de tests                                  | Plan de tests                  | Stratégie et types de tests                 |
|                                    | Rapports de tests                              | Rapport de validation          | Résultats et corrections des tests          |





# DOSSIER TECHNIQUE – PGI Gestion de Vente de Vélos

## 1. Vue d'ensemble

### 1.1 Contexte

Le PGI a été développé pour remplacer les outils fragmentés utilisés par l'entreprise (Excel, applications indépendantes).

Objectif : **centraliser la gestion complète d'un magasin de vélos**, incluant plusieurs modèles, et variations (tailles, couleurs).

### 1.2 Modules couverts

- **Ventes** : gestion clients, nouvelle commande.
- **Achats** : gestion fournisseurs, commandes d'approvisionnement.
- **Stocks** : vélos, mouvements.
- **Facturation** : factures PDF.
- **Ressources humaines** : employés, paie.
- **Dashboard** : rapports, graphiques.

## 2. Architecture technique

### 2.1 Architecture 3 tiers

1. **Présentation** : HTML5 – CSS3 – JavaScript ES6 – Bootstrap 5
2. **Métier** : PHP 8 – Apache 2.4 (AMPPS / WAMP / XAMPP)
3. **Données** : MySQL 8.0

### 2.2 Pattern MVC

- **Model** : classes PHP pour l'accès aux tables (produits, ventes, etc.)
- **View** : pages HTML/PHP + composants Bootstrap
- **Controller** : scripts orchestrant la logique et les routes

## 2.3 Stack technique

| Composant       | Technologie             |
|-----------------|-------------------------|
| Langage         | PHP 8.x                 |
| Base de données | MySQL 8                 |
| Frontend        | HTML/CSS/JS + Bootstrap |
| IDE             | Visual Studio Code      |
| Versionnement   | GitHub                  |

## 3. Base de données

### 3.1 Structure principale

#### Ventes

ID, Client, Date, Total (€), Statut, Actions

#### Achats

ID, Fournisseur, Date, Montant total, Statut, Actions

#### Produits

Référence, Nom, Marque, Prix vente, Stock, Actions

#### Facturation

ID, Client, Commande, Date, Montant, Actions

#### RH

ID, Prénom, Nom, Email, Poste, Salaire, Date d'embauche, Actions

### 3.2 Optimisations

- **Vues SQL** : v\_stock\_disponible, v\_ventes\_mois, v\_alertes\_stock
- **Triggers** : mise à jour du stock lors d'une vente ou réception fournisseur
- **Procédures stockées** : creer\_vente(), maj\_stock(), add\_mouvement\_stock()
- **Indexation** sur produits, codes articles, emails, dates.

## 4. Développement Backend

### 4.1 Exemple de connexion PDO

```
// include/db.php

define('DB_HOST', 'localhost');

define('DB_NAME', 'pgi_velos');

define('DB_USER', 'root');

define('DB_PASS', '');

$pdo = new PDO(

    "mysql:host=".DB_HOST.";dbname=".DB_NAME.";charset=utf8mb4",

    DB_USER, DB_PASS,

    [

        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,

        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC

    ]

);
```

## 5. Fonctionnalités par module

### 5.1 Module Ventes

- Gestion des clients (particulier / pro)
- Vérification de stock automatique



## **5.2 Module Achats**

- Bons de commande fournisseur
- Statuts : brouillon → envoyé → reçu → clôturé
- Réception partielle ou totale
- Mise à jour stock automatique

## **5.3 Module Produits**

- Gestion des vélos
- Multi-tailles
- Alertes stock bas
- Inventaire annuel ou mensuel

## **5.4 Module Facturation**

- Factures PDF

## **5.5 Module RH**

- Gestion employés
- Calcul paie automatique
- Export fiche de paie PDF

## **6. Sécurité**

### **Authentification**

- Hachage bcrypt
- Sessions sécurisées
- Rôles : admin, manager, vendeur, employé, fournisseur

## Protection

- Requêtes préparées
- Vérification rôle → accès

## 7. Tests et performances

### 7.1 Tests fonctionnels

| Module   | Test                    | Résultat |
|----------|-------------------------|----------|
| Ventes   | création client + vente | OK       |
| Achats   | Nouvel achat            | OK       |
| Produits | inventaire              | OK       |
| Factures | génération PDF          | OK       |
| RH       | Nouvel employé          | OK       |

### 7.2 Performance

| Opération      | Objectif | Résultat |
|----------------|----------|----------|
| Dashboard      | < 600 ms | OK       |
| Liste produits | < 300 ms | OK       |
| Création vente | < 1 s    | OK       |

## 8. Maintenance

### 8.1 Sauvegardes automatiques

- Dump MySQL quotidien
- Conservation : 7 jours + 4 hebdomadaires
- Restauration testée chaque semaine

## 8.2 Nettoyage mensuel

```
DELETE FROM sessions
```

```
WHERE last_activity < DATE_SUB(NOW(), INTERVAL 30 DAY);
```

```
OPTIMIZE TABLE velos, stock, ventes;
```

## 9. Évolutions prévues

### Court terme

- Inventaire mobile (QR code)
- Module comptabilité simplifiée

### Moyen terme

- Synchronisation e-commerce (API REST)
- Outil d'analyse ventes (IA)

### Long terme

- Migration vers framework Laravel / Symfony
- Application mobile vendeur
- Multi-boutiques

## 10. Ressources

### Accès système

- URL locale : <https://pgivelo.site/>

# Étude de faisabilité

## 1. Présentation générale

**Nom du projet :** Les fous du guidon ! – PGI pour la gestion intégrée d'une PME de vente de vélos

**Équipe projet :** Kenza Chahnaz et Ngueba Anne-Maéra

**Encadrant pédagogique :** CHERGUI Anis

**Durée du projet :** 3 semaines

**Outils principaux :** Laravel, PHP, MySQL, XAMPP, GitHub

### Objectif:

Concevoir et développer un PGI (Progiciel de Gestion Intégré) qui centralise les activités d'une PME spécialisée dans la vente et la maintenance de vélos. Le système doit regrouper les fonctions clés : ventes, achats, gestion des stocks et gestion RH, afin d'améliorer la productivité et le suivi des opérations.

## 2. Faisabilité technique

| <u>Élément</u>                        | <u>Description</u>                                                   |
|---------------------------------------|----------------------------------------------------------------------|
| <b>Langage principal</b>              | PHP 8 avec le framework <b>Laravel</b>                               |
| <b>Base de données</b>                | <b>MySQL</b> , intégrée à XAMPP                                      |
| <b>Serveur local</b>                  | XAMPP (Apache + MySQL)                                               |
| <b>Environnement de développement</b> | Visual Studio Code                                                   |
| <b>Front-end</b>                      | Blade (moteur de templates Laravel) + CSS/Bootstrap                  |
| <b>Hébergement</b>                    | Test local, puis hébergement en ligne (ex : 000webhost ou Hostinger) |
| <b>Gestion de version</b>             | Git / GitHub pour le suivi collaboratif                              |
| <b>Système d'exploitation</b>         | MacOS Sequoia                                                        |

### Conclusion technique :

Toutes les technologies nécessaires sont gratuites, bien documentées et compatibles entre elles. L'équipe maîtrise les bases de Laravel et MySQL → le développement est techniquement réalisable.

### 3. Faisabilité économique

| <u>Ressources</u>              | <u>Coût (€)</u> | <u>Détails</u>                     |
|--------------------------------|-----------------|------------------------------------|
| Ordinateurs personnels         | 0               | Ressources des membres de l'équipe |
| XAMPP                          | 0               | Gratuit                            |
| Laravel / PHP / MySQL          | 0               | Open source                        |
| Hébergement local              | 0               | Environnement local                |
| Autres (électricité, internet) | Négligeable     | Utilisation personnelle            |

Budget total estimé : 0 €. Le projet ne requiert aucun investissement financier majeur. Il repose entièrement sur des outils gratuits et open source.

### 4. Faisabilité organisationnelle

| <u>Facteur</u>         | <u>Description</u>                                                   |
|------------------------|----------------------------------------------------------------------|
| Effectif               | 2                                                                    |
| Compétences            | Bases en PHP, MySQL, conception UML, et développement web            |
| Répartition du travail | - Kenza : code<br>- Anne-Maéra : livrables                           |
| Encadrement            | Professeur CHERGUI Anis                                              |
| Communication          | Réunions de suivi, Trello ou Google Drive pour la gestion des tâches |

**Durée : 3 semaines**

Organisation par phases :

1. Analyse et modélisation
2. Conception et base de données
3. Développement des modules
4. Tests et validation

**Conclusion organisationnelle :**

Le projet est **réalisable dans le délai imparti** avec une bonne répartition des tâches.

## 5. Faisabilité globale

| <u>Type de faisabilité</u> | <u>Résultat</u> | <u>Commentaire</u>                                |
|----------------------------|-----------------|---------------------------------------------------|
| <b>Technique</b>           | oui             | Technologies maîtrisées et compatibles            |
| <b>Économique</b>          | oui             | Coût nul grâce à l'utilisation d'outils gratuits  |
| <b>Organisationnelle</b>   | oui             | Travail en binôme équilibré et encadré            |
| <b>Temporelle</b>          | oui             | Délai court mais gérable avec bonne planification |

**Conclusion générale :**

Le projet PGI "VeloMax" est **totalelement faisable**.

Les conditions techniques, économiques et humaines sont réunies pour garantir un développement réussi et une démonstration fonctionnelle du système lors de la soutenance.

# Plan de gestion de projet – PGI VeloMax

## 1. Présentation générale

| <u>Élément</u>          | <u>Détails</u>                                                                                                          |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Nom du projet           | PGI VeloMax                                                                                                             |
| Objectif principal      | Concevoir et développer un PGI pour une PME de vente de vélos, intégrant les fonctions de ventes, achats, stocks et RH. |
| Durée totale            | 3 semaines                                                                                                              |
| Équipe projet           | Kenza Chahnaz & Ngueba Anne-Maéra                                                                                       |
| Encadrant pédagogique   | CHERGUI Anis                                                                                                            |
| Méthode de gestion      | Approche en cascade (Waterfall) adaptée à un projet court et académique                                                 |
| Environnement technique | Laravel, PHP, MySQL, XAMPP, Blade, Bootstrap, VS Code, GitHub                                                           |

## 2. Objectifs opérationnels

- Développer une **solution complète et intégrée** pour gérer les activités clés d'une PME.
- Créer une **interface utilisateur ergonomique** pour simplifier la gestion des données.
- Garantir une **structure de base de données fiable et cohérente**.
- Tester et valider le fonctionnement du PGI avant sa présentation.

### 3. Organisation du travail

| Rôle          | Membres                               | Responsabilités principales                                 |
|---------------|---------------------------------------|-------------------------------------------------------------|
| Développeuses | Kenza Chahnaz et<br>Ngueba Anne-Maéra | Analyse, conception, développement,<br>tests, documentation |
| Encadrant     | CHERGUI Anis                          | Validation, suivi pédagogique                               |

### 4. Découpage des phases du projet

| <u>Phase</u>            | <u>Durée<br/>estimée</u> | <u>Description / Objectif</u>                                             | <u>Livrables associés</u>                                           |
|-------------------------|--------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------|
| Analyse et<br>cadrage   | 3 jours                  | Identifier les besoins,<br>objectifs, contraintes et<br>acteurs           | Cahier des charges, étude<br>de faisabilité                         |
| Conception              | 4 jours                  | Définir les modèles de<br>données, diagrammes UML,<br>et maquettes        | Schéma de base de<br>données, diagrammes<br>UML, maquettes d'écrans |
| Développement           | 8 jours                  | Implémenter les modules<br>(ventes, achats, stocks, RH)<br>avec Laravel   | Code source du PGI                                                  |
| Tests et<br>validation  | 3 jours                  | Vérifier le bon<br>fonctionnement du système<br>et corriger les anomalies | Rapport de tests,<br>validation fonctionnelle                       |
| Livraison<br>soutenance | 2 jours                  | Préparer la soutenance,<br>documentation et démo                          | Rapport final, diaporama,<br>démonstration                          |

### 5. Diagramme de Gantt simplifié (3 semaines)

| Phase                     | Semaine 1           | Semaine 2           | Semaine 3           |
|---------------------------|---------------------|---------------------|---------------------|
| Analyse & faisabilité     | ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ |                     |                     |
| Conception & modélisation | ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ | ■                   |                     |
| Développement des modules |                     | ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ | ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ |



| Phase                  | Semaine 1 | Semaine 2 | Semaine 3 |
|------------------------|-----------|-----------|-----------|
| Tests & validation     |           |           |           |
| Préparation soutenance |           |           |           |

**Durée totale** : 15 jours ouvrés (3 semaines). Les phases peuvent légèrement se chevaucher selon l'avancement réel.

## 6. Gestion des risques

| <u>Risque potentiel</u>             | <u>Probabilité</u> | <u>Impact</u> | <u>Mesure préventive</u>                               |
|-------------------------------------|--------------------|---------------|--------------------------------------------------------|
| Difficultés techniques avec Laravel | Moyenne            | Moyen         | Suivi de la documentation Laravel et entraide binôme   |
| Retard de développement             | Moyenne            | Élevé         | Planification stricte et répartition claire des tâches |
| Problèmes de compatibilité MySQL    | Faible             | Moyen         | Tests réguliers sur serveur local                      |
| Perte de données ou de code         | Moyenne            | Élevé         | Sauvegardes fréquentes sur GitHub                      |
| Manque de temps pour les tests      | Moyenne            | Moyen         | Prévoir une marge de 2 jours pour correction           |

## 7. Suivi et communication

| <u>Méthode</u>       | <u>Fréquence</u> | <u>Outil</u>          |
|----------------------|------------------|-----------------------|
| Réunion d'équipe     | 2 fois/ semaine  | Présentiel / en ligne |
| Validation encadrant | 1 fois/ semaine  | Présentiel            |
| Sauvegardes de code  | Quotidien        | GitHub                |

## **8. Livrables attendus**

- Cahier des charges
- Étude de faisabilité
- Plan de projet (présent document)
- Diagrammes UML
- Base de données MySQL
- Code source Laravel du PGI
- Documentation technique et utilisateur
- Rapport de tests et rapport final
- Présentation de soutenance

## **9. Conclusion**

Le plan de gestion du projet PGI VeloMax offre une vision claire et structurée de la démarche à suivre. Avec une bonne coordination entre les deux membres du binôme et un suivi, même minime de l'encadrant, le projet est parfaitement réalisable dans les délais et garantira un PGI fonctionnel, ergonomique et adapté à la gestion d'une PME.

# RAPPORT DE TESTS

## 1. Objectif du rapport

Le but du rapport est de vérifier que l'ensemble des modules du PGI fonctionne correctement, sans régressions, selon les spécifications définies dans le cahier des charges et les SFG/SFD.

Ce rapport couvre :

- les tests fonctionnels,
- les tests techniques,
- les tests de performance,
- les anomalies détectées et leur statut.

## 2. Environnement de test

| Élément               | Valeur              |
|-----------------------|---------------------|
| Serveur               | Apache 2.4 (AMP/PS) |
| PHP                   | 8.0+                |
| Base de données MySQL | 8                   |
| Navigateur            | Chrome / Edge       |

## 3. Tests fonctionnels

### 3.1 Résultat global

Tous les modules principaux ont passé les tests de base attendus.

| Module            | Cas de test                                              | Résultat |
|-------------------|----------------------------------------------------------|----------|
| Ventes            | Création client → création vente → déstockage            | OK       |
| Achats            | Création bon de commande → réception → mise à jour stock | OK       |
| Produits / Stocks | Inventaire, alertes stock bas, mouvement stock           | OK       |

| Module      | Cas de test                  | Résultat |
|-------------|------------------------------|----------|
| Facturation | Génération PDF d'une facture | OK       |
| RH          | Ajout employé                | OK       |
| Dashboard   | Chargement des KPI mensuels  | OK       |

#### 4. Tests détaillés par module

##### 4.1 Module Ventes

| Test                   | Attendu                     | Résultat |
|------------------------|-----------------------------|----------|
| Nouveau client         | Le client apparaît bien     | OK       |
| Création d'une vente   | Calcul total, TVA corrects  | OK       |
| Vérification stock     | Erreur si stock insuffisant | OK       |
| Déstockage automatique | Stock décrémenté            | OK       |

**Aucune anomalie détectée.**

##### 4.2 Module Achats

| Test                                          | Attendu | Résultat |
|-----------------------------------------------|---------|----------|
| Création bon de commande Statut « brouillon » |         | OK       |

**Note : réception partielle testée avec succès.**

##### 4.3 Module Produits

| Test                                | Attendu          | Résultat |
|-------------------------------------|------------------|----------|
| Ajout vélo                          | Référence unique | OK       |
| Alerte stock bas                    | Valeur correcte  | OK       |
| Mouvements stock Historique complet |                  | OK       |

#### 4.4 Module Facturation

| Test                                    | Attendu | Résultat |
|-----------------------------------------|---------|----------|
| Génération PDF Numérotation automatique |         | OK       |

#### 4.5 Module RH

| Test                               | Attendu | Résultat |
|------------------------------------|---------|----------|
| Ajout employé Comptes + rôle actif |         | OK       |

### 5. Tests de performance

| Action testée          | Temps attendu | Temps mesuré | Résultat |
|------------------------|---------------|--------------|----------|
| Chargement Dashboard   | < 600 ms      | 420 ms       | OK       |
| Liste produits         | < 300 ms      | 240 ms       | OK       |
| Création d'une vente   | < 1 s         | 710 ms       | OK       |
| Génération facture PDF | < 2 s         | 1,2 s        | OK       |
| Consultation employés  | < 500 ms      | 300 ms       | OK       |

**Tous les objectifs ont été atteints.**

### 6. Tests de sécurité

| Test          | Attendu                   | Résultat |
|---------------|---------------------------|----------|
| Injection SQL | Tentatives bloquées (PDO) | OK       |

| Test                      | Attendu                | Résultat |
|---------------------------|------------------------|----------|
| Accès non autorisé        | Redirection vers login | OK       |
| Mot de passe non chiffré  | Impossible             | OK       |
| Accès restreints par rôle | Filtré correctement    | OK       |

Les rôles ont été testés. Toutes les restrictions fonctionnent.

## 7. Anomalies détectées

| ID  | Description                                       | Gravité | Statut   |
|-----|---------------------------------------------------|---------|----------|
| A01 | Mauvais alignement dans PDF facture               | Mineure | Corrigé  |
| A02 | Alerte stock bas envoyée 2 fois                   | Mineure | Corrigé  |
| A03 | Temps de réponse lent sur liste achats (> 800 ms) | Moyenne | Optimisé |

→ **Aucune anomalie bloquante au moment du rapport.**

## 8. Validation finale

Tous les modules principaux fonctionnent.

Les performances sont conformes.

Les fonctionnalités critiques (ventes, achats, facturation) sont stables.

# SFD – Spécifications Fonctionnelles Détaillées

## 1. Architecture technique

- **Front-end** : Blade, HTML, CSS, JS
- **Back-end** : Laravel (PHP)
- **Base de données** : MySQL
- **Modèle MVC** Laravel
- **Middleware** : rôles utilisateurs

## 2. Détail des modules

### 2.1 Module Dashboard

| Fonction            | Entrées | Traitements        | Sorties              |
|---------------------|---------|--------------------|----------------------|
| Affichage stats     | -       | Requêtes de calcul | Chiffres, graphiques |
| Produits en rupture | Stock   | Filtrage < seuil   | Liste d'alertes      |

### 2.2 Module Clients

| Fonction          | Entrées    | Traitements                | Sorties          |
|-------------------|------------|----------------------------|------------------|
| Ajouter client    | Formulaire | Validation + insertion SQL | Confirmation     |
| Modifier client   | ID client  | Update SQL                 | Message succès   |
| Historique achats | ID client  | Jointure ventes-produits   | Liste des achats |

### 2.3 Module Produits

| Fonction          | Entrées         | Traitements                | Sorties           |
|-------------------|-----------------|----------------------------|-------------------|
| Ajouter produit   | Données produit | Insertion SQL              | Message           |
| Mise à jour stock | Quantité        | Update table stock         | Nouvel état stock |
| Alerte seuil bas  | -               | SELECT WHERE stock < seuil | Notification      |

## 2.4 Module Fournisseurs

| Fonction             | Entrées        | Traitements      | Sorties      |
|----------------------|----------------|------------------|--------------|
| Ajout fournisseur    | Formulaire     | Insertion SQL    | Confirmation |
| Historique commandes | ID fournisseur | SELECT commandes | Tableau      |

## 2.5 Module Achats

| Fonction              | Entrées                | Traitements     | Sorties          |
|-----------------------|------------------------|-----------------|------------------|
| Créer bon de commande | Produits + fournisseur | Insert commande | Document         |
| Validation réception  | Quantité reçue         | Update stock    | Stock mis à jour |
| Paiement achat        | ID achat               | Update statut   | Confirmation     |

## 2.6 Module Ventes

| Fonction    | Entrées           | Traitements                 | Sorties |
|-------------|-------------------|-----------------------------|---------|
| Créer vente | Client + produits | Calcul total + update stock | Facture |
| Historique  | Date / client     | SELECT filtré               | Tableau |

## 2.7 Module Factures

| Fonction            | Entrées    | Traitements        | Sorties      |
|---------------------|------------|--------------------|--------------|
| Générer facture     | ID vente   | PDF Laravel domPDF | Facture PDF  |
| Marquer comme payée | ID facture | Update SQL         | Confirmation |

## 2.8 Module RH

| Fonction             | Entrées    | Traitements            | Sorties        |
|----------------------|------------|------------------------|----------------|
| Ajouter employé      | Données RH | Insert SQL             | Confirmation   |
| Enregistrer présence | ID employé | Insert entrée / sortie | Fiche présence |



| <b>Fonction</b> | <b>Entrées</b> | <b>Traitements</b>  | <b>Sorties</b> |
|-----------------|----------------|---------------------|----------------|
| Suivi absences  | -              | Analyse calendriers | Rapport        |

# SFG – Spécifications Fonctionnelles Générales

Ce document décrit **ce que doit faire le système** sans entrer dans les détails techniques.

Il sert à montrer la vision globale du fonctionnement du futur PGI.

## 1. Objectif du SFG

Décrire les fonctionnalités principales du PGI de gestion des ventes de vélos, afin de s'assurer que les besoins du client (la PME) soient bien compris.

## 2. Contexte

La société " **Les fous du guidon !** " souhaite un système centralisé pour gérer ses ventes, ses clients, son stock et sa facturation.

Le PGI devra permettre une meilleure traçabilité et un suivi en temps réel de l'activité.

## 3. Fonctionnalités attendues

### a) Gestion des produits

- Ajouter / modifier / supprimer un vélo (référence, modèle, prix, quantité, fournisseur).
- Consulter l'état du stock.
- Alerte automatique en cas de stock faible.

### b) Gestion des clients

- Enregistrer les informations d'un client (nom, prénom, adresse, téléphone, e-mail).
- Rechercher un client dans la base.
- Historique des achats.

### c) Gestion des ventes

- Enregistrer une nouvelle vente.
- Calcul automatique du montant total.
- Génération de facture.
- Mise à jour automatique du stock après-vente.

### d) Gestion des utilisateurs

- Accès sécurisé par identifiant et mot de passe.
- Rôles : administrateur, vendeur, comptable.
- Journal des connexions et des opérations.

#### **e) Tableau de bord**

- Statistiques de ventes (par jour, semaine, mois).
- Graphiques sur les produits les plus vendus.
- Indicateurs de performance (CA, bénéfices, stock restant).

#### **4. Contraintes**

- Application accessible depuis un navigateur web.
- Base de données centralisée.
- Interface simple et intuitive.

#### **5. Résultats attendus**

- Réduction du temps de traitement des ventes.
- Moins d'erreurs de saisie.
- Meilleure visibilité sur la performance commerciale.